Лабораторная работа №8

Тема: Подключение к базе данных PostgreSQL в MS Visual Studio 2019

Цель: Получить практические навыки разработки, тестирования, и построения базовой версии приложения Windows для работы с базой данных.

Объем: 6 академических часов (1,5 недели).

Задание

1. Создать в MS Visual Studio проект типа Windows Forms.
2. Создать подключение к базе данных, разработанной при выполнении лабораторной работы №5. Обеспечить видимость соединения во всех формах приложения.
3. В главной форме создать основное меню, состоящее из 5 пунктов:
   – «Справочники» – отображение справочных данных;
   – «Данные» – отображение оперативных данных;
   – «Отчеты» – вывод результатов запросов к представлениям, созданным в лабораторной работе №6, которые отражают модель черный ящик из лабораторной работы №1;
   – «О программе» – выдача справки и данных о приложении и разработчиках;
   – «Выход» – выход из приложения.
4. Создать формы для отображения справочников. Каждую форму связать с соответствующим подпунктом главного меню «Справочники».
5. Создать формы для отображения оперативных данных. Каждую форму связать с соответствующим подпунктом главного меню «Данные».
6. Разработать статическую форму сведений о программе и связать ее с соответствующим пунктом меню.
7. Выполнить описание разработанных форм приложения в виде табл. 8.1.

Таблица 8.1. Перечень разработанных компонентов приложения

№ пп	Имя формы	Описание

В отчет включить:

- перечень разработанных форм в виде таблицы 8.1;
- изображения разработанных форм приложения в режиме запуска **с полученными из базы данных тестовыми данными**.
- изображение обозревателя решений с развернутым проектом;
- листинги кода модуля Program.cs и всех разработанных форм.

Методические указания к выполнению

Для того, чтобы интерфейс был удобен для использования конечными пользователями, наиболее удачным вариантом будет создания Windows-приложения, состоящего из отдельных форм, связанных с одной основной формой, из которой они будут вызываться. Для этого создается типичное приложение Windows.

Подключение к базы данных

Для подключения к базе данных PostgreSQL можно использовать дополнительный NuGet пакет Npgsql, который можно установить через диспетчер пакетов (рис. 8.1).

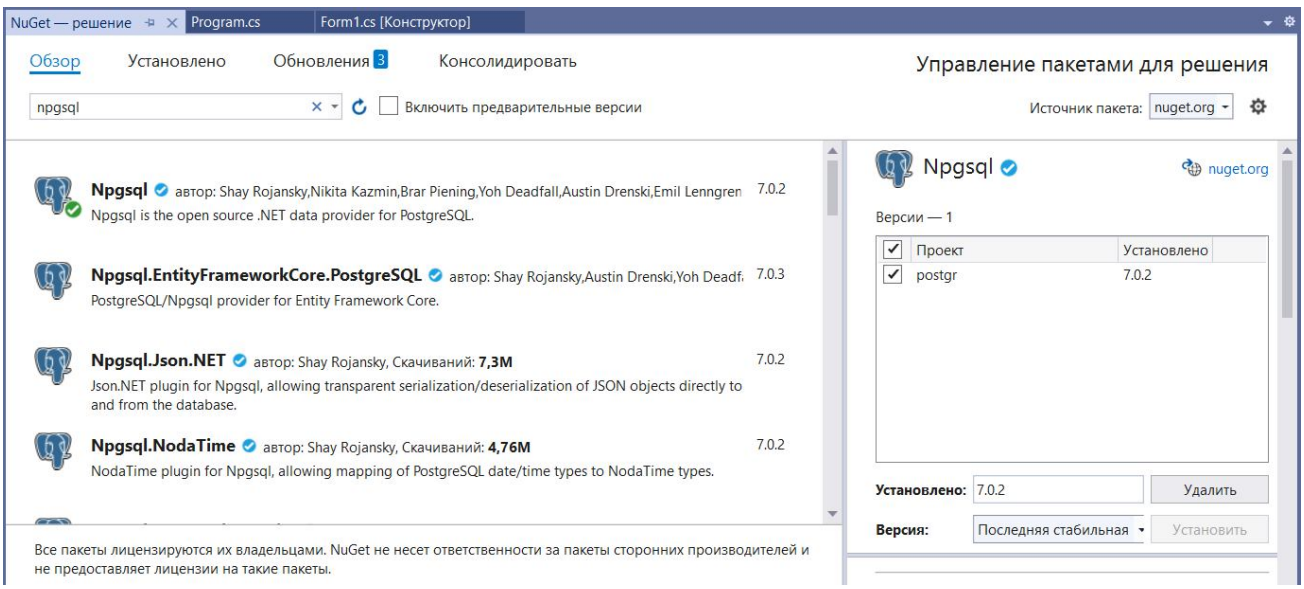

Рисунок 8.1 – Загрузка пакета Npgsql.

Пример приложения, которое может подключаться к базе данных PostgreSQL сервера и запрашивать информацию из таблиц и представлений можно найти по ссылке: https://github.com/Mistral-war2ru/PG-connect.

После установки пакета необходимо подключить его, прописав в Program.cs строку using Npgsql;

Для реализации подключения и обработки запросов необходимо создать отдельный класс, в котором создать статичные методы для запросов и статичное поле для хранения состояния подключения.

```
private static NpgsqlConnection con = null;
```

Подключение к базе данных создается с помощью создания нового экземпляра класса NpgsqlConnection, конструктор которого принимает строку подключения. Рекомендуется сразу подготовить метод для создания нового подключения с минимальными необходимыми параметрами Host, Port, User, Password, DataBase. Созданный метод сначала должен проверять наличие уже открытого подключения к базе данных, завершать его и только потом открывать новое.

```
public static void OpenConnection(string host, string port, string user, string pass, string db)
{
    if (con != null)
    {
        if (con.State == ConnectionState.Open) con.Close();
        con.Dispose();
    }
    con = new NpgsqlConnection(@"Server=" + host + ";Port=" + port + ";User Id=" + user + ";Pass
    con.Open();
    if (con.State == ConnectionState.Open) MessageBox.Show("Connected!");
    else MessageBox.Show("NOT Connected!");
}
```

В примере, представленном по ссылке выше, все параметры могут быть заданы с помощью TextBox, что позволяет подключаться к разным серверам и базам данных не выходя из приложения (рис. 8.2).

```
private void button1_Click(object sender, EventArgs e)
{
    try
    {
        PG.OpenConnection(textBox1.Text, textBox2.Text, textBox3.Text, textBox4.Text, textBox5.Text);
    }
    catch(Exception ex) { MessageBox.Show("Error!\n" + ex.Message); }
}
```

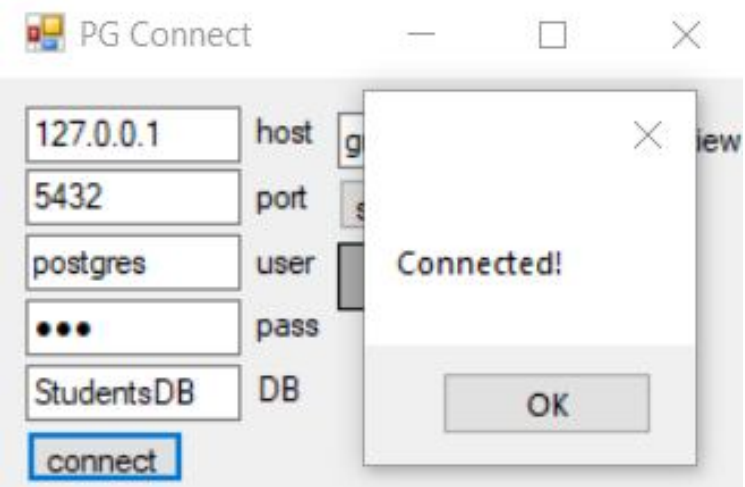

Рисунок 8.2 – Подключение к базе данных.

Для отображения данных в приложения будем использовать элемент типа DataGridView, который нужно найти в панели элементов и перетащить на форму. При этом следует убрать возможность изменения и не выбирать источник данных (рис. 8.3).

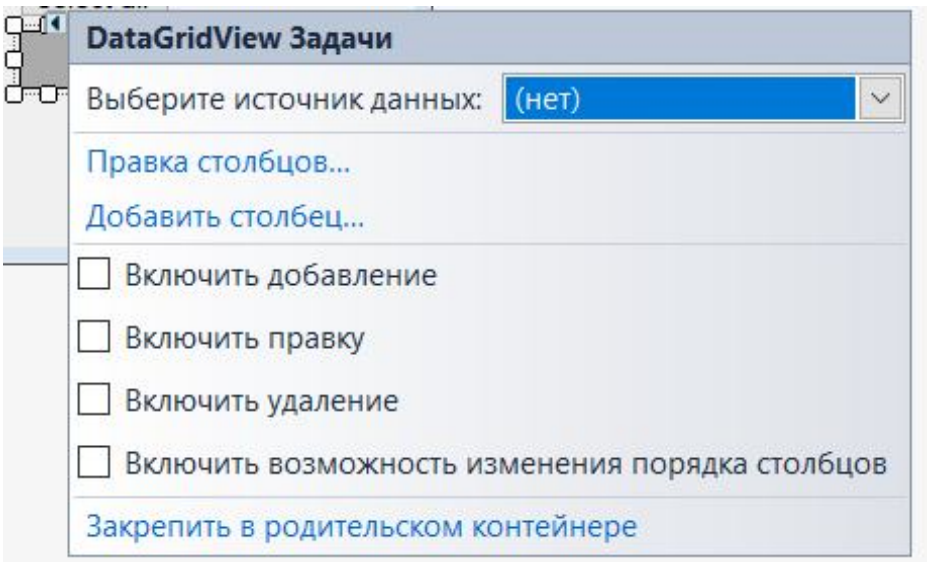

Рисунок 8.3 – Настройка DataGridView.

Так как Visual Studio по умолчанию не поддерживает PostgreSQL как источник данных без установки дополнительных расширений и плагинов, то для того, чтобы в сетке появились данные их необходимо будет заполнить вручную, или же по-другому этот процесс называется “заселение сетки” (populate DataGrid).

Чтобы получить данные, которые можно будет отобразить на сетку в примере для открытого подключения создаётся команда.

```
NpgsqlCommand com = con.CreateCommand();
com.CommandText = "Select * from public." + table_view;
```

С помощью такого построения текста команды можно создать универсальный запрос который получает все данные из указанной таблицы или представления.

Для выполнения запроса используется ExecuteReader, который идеально подходит для запросов типа SELECT, так как обратно возвращается цельный набор данных.

```
NpgsqlDataReader dt = com.ExecuteReader(CommandBehavior.Default);
```

Рекомендуется дополнительно обработать полученный набор данных и представить в виде упорядоченной структуры данных, например, в виде списка словарей или списка списков. В примере используется второй вариант, при этом так как запрос должен быть универсальным и названия столбцов заранее неизвестны, первый список не содержит самих данных, вместо этого в нём находятся имена столбцов.

```
bool add_names = false;
while (dt.Read())
{
    try
    {
        if (!add_names)
        {
            List<string> list_names = new List<string>();
            for (int i = 0; i < dt.FieldCount; i++)
            {
                list_names.Add(dt.GetName(i));
            }
            list.Add(list_names);
            add_names = true;
        }
        List<string> inside_list = new List<string>();
        for (int i = 0; i < dt.FieldCount; i++)
        {
            inside_list.Add(dt.GetValue(i).ToString());
        }
        list.Add(inside_list);
    }
    catch (Exception ex) { MessageBox.Show(ex.Message); }
```

Важно не забыть закрыть процесс чтения после завершения.

dt.Close();

При вызове метода, например, по нажатии кнопки, будет получен структурированный список списков строк данными из которого легко заселить сетку отображения. Для этого сначала сетка очищается от старых данных.

List<List<string>> list_list = PG.Select_all(“Groups”);

dataGridView1.Rows.Clear();

dataGridView1.Columns.Clear();

Прежде всего сначала заполняются названия столбцов, которые в примере хранятся в первом списке.

```
foreach (string ss in list_list[0])
{
    dataGridView1.Columns.Add(ss, ss);
}
```

Затем уже выполняется само заселение сетки данными. Так как первый список использовался для хранения имен столбцов, он пропускается. Все

остальные списки переводятся в массив строк и добавляются в хранилище данных.

```
bool skip1 = false;
foreach (List<string> list in list_list)
{
    if (skip1)
    {
        string[] v = new string[list.Count];
        int k = 0;
        foreach (string ss in list)
        {
            v[k] = ss;
            k++;
        }
        dataGridView1.Rows.Add(v);
    }
    skip1 = true;
}
```

Пример работы приложения на (рис. 8.4) и сравнение результата с данными из самой базы данных (рис. 8.5).

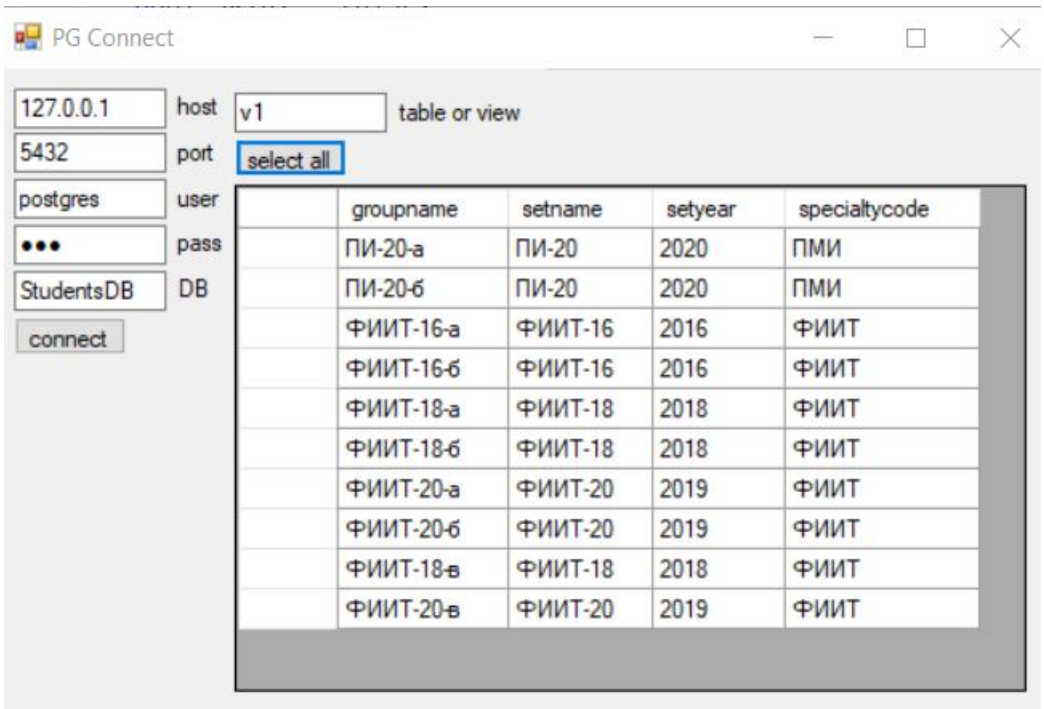

Рисунок 8.4 – Пример работы приложения

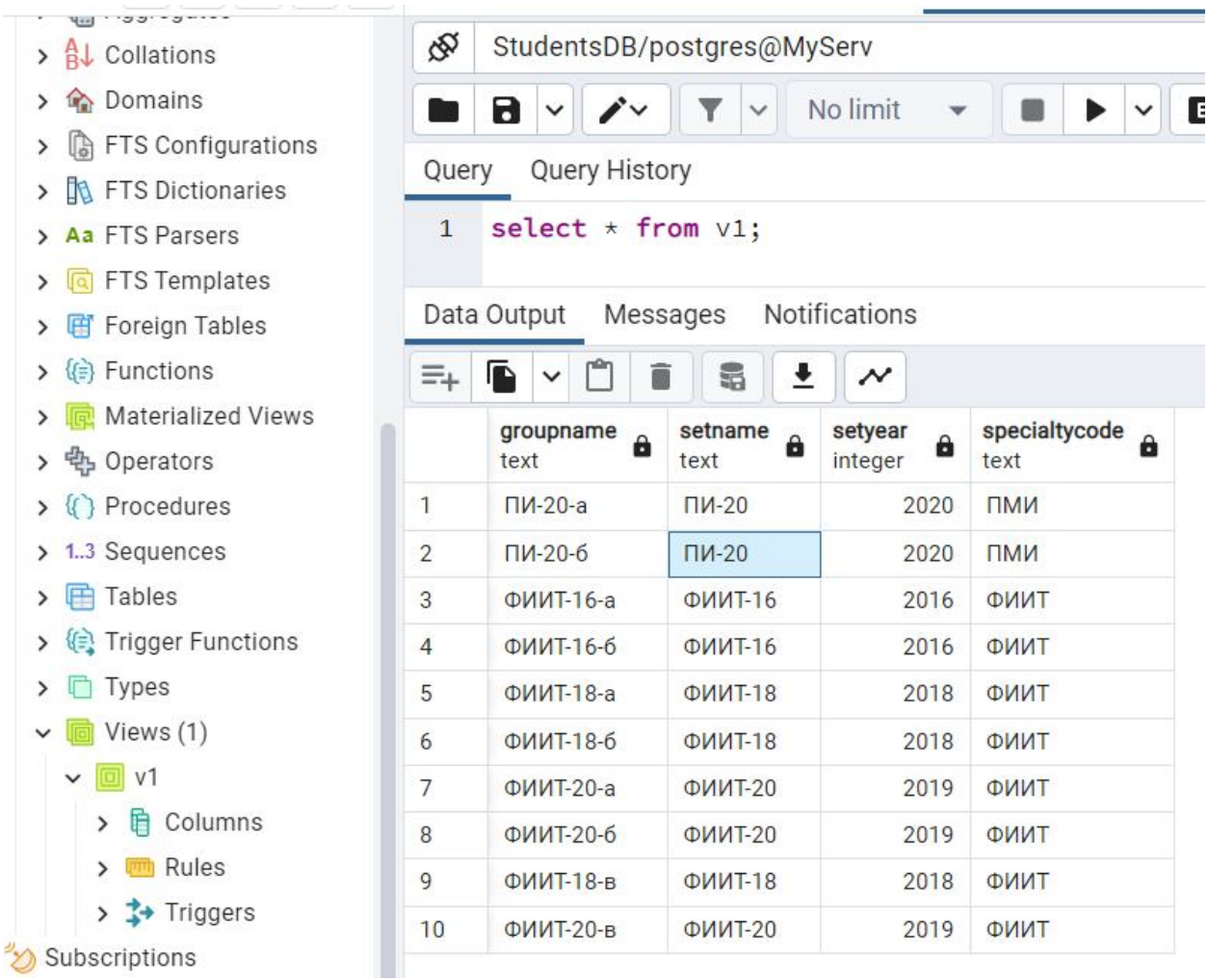

Рисунок 8.5 – Данные в базе.

Создание меню

Основное меню состоит из 5 пунктов:

- вывода справочных таблиц;
- вывода оперативных таблиц;
- генерации отчетов;
- выдачи справки и данных о разработчиках;
- пункта выхода.

Для создания такого меню необходимо выполнить ряд действий. Для начала выбрать в меню Вид → Панель элементов → MenuStrip (см. рис. 8.6) и перенести его на форму.

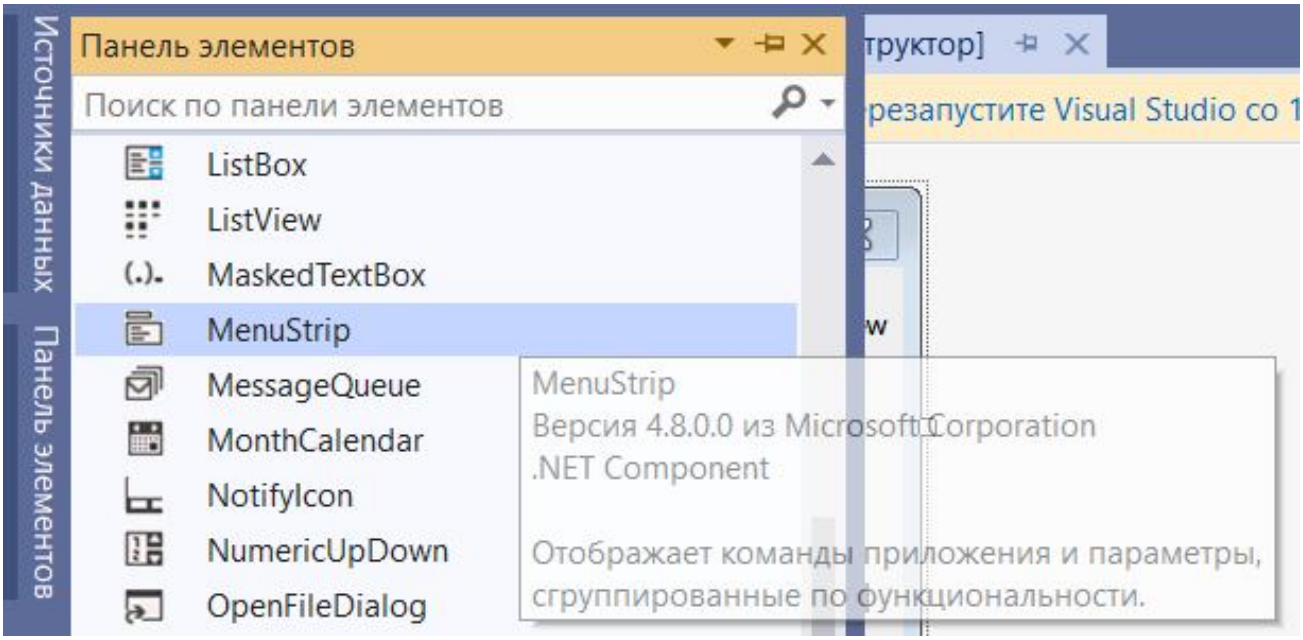

Рисунок 8.6 – Выбор инструмента на панели инструментов VS

Следующим действием является заполнение пунктов меню. Для этого используется конструктор, показанный на рис. 8.7.

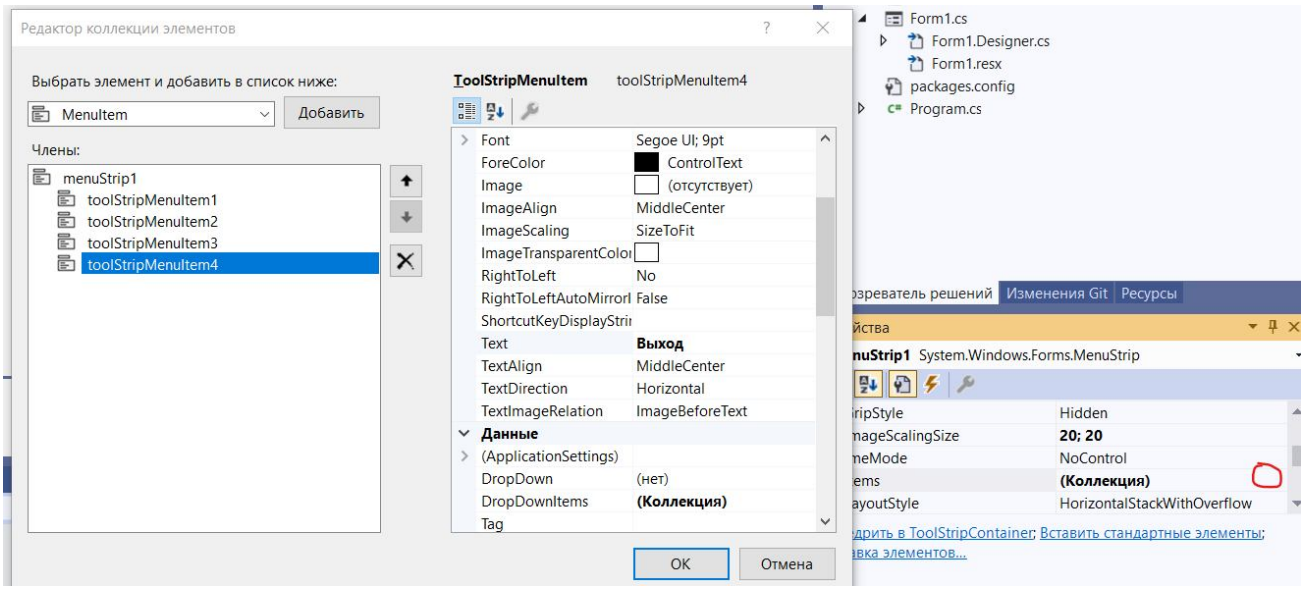

Рисунок 8.7 – Конструктор инструмента MenuStrip

При двойном нажатии на одном из введенных пунктов меню открывается код формы, а в нем обработчик события выбора этого пункта. В нем следует прописать связь основной формы с вызываемой. Это делается так:

```
private void продукцияToolStripMenuItem_Click(object sender,
EventArgs e)
        {
            // Создается объект формы
            Products products = new Products();
            // Вызывается данная форма
            products.Show();
        }
```

Здесь:

`Products` – это имя формы в обозревателе решений.

`products` – это объектная переменная.

`Products products = new Products();` –инициализация этой переменной.

`products.Show();` – это вызов метода для отображения формы, на которую ссылается объектная переменная `products`, на экране.

Для обработчика закрытия вызываемых форм используется 2 метода:

1) `Close();` // Для закрытия текущей формы
2) `Application.Exit();` // Для закрытия всего приложения

```
private void toolStripMenuItem5_Click(object sender, EventArgs e)
{
    Application.Exit();
}
```

Создание формы для работы с таблицами

Создание новой формы производится нажатием Проект → Добавить форму Windows…

Либо можно в обозревателе решений на значке проекта вызвать контекстное меню и последовательно выбрать Добавить → Форма Windows….

При создании вызываемой формы с данными по отдельным таблицам, кроме выпадающего меню, потребуется компонент DataGridView.

На дополнительные формы необходимо добавить свои меню с пунктами возврата и выхода. Для пунктов меню «Вернуться» и «Выйти из проекта» следует прописать следующий код:

```
private void вернутToolStripMenuItem_Click(object sender, EventArgs e)
    {
        this.Close();
        //эта функция при закрытии формы будет выводить
          на первый план главную форму
        Program.MainForm.Activate();
    }
private void выйтиToolStripMenuItem_Click(object sender, EventArgs e)
    {
        Application.Exit();
    }
```

Теперь можно возвращаться на главную форму и закрывать проект из любой формы.